# Claude Code for Empirical Research

A Hands-On Session

John Barrios · Yale School of Management

Prepared for Stanford PhD students
Tuesday, November 3, 2026 · 12:00–3:00 pm · Room E247

# Today: Three Hours, Mostly Hands-On

| Time | Block | You leave with |
|---|---|---|
| 12:00–12:20 | Framing | how the agent works |
| 12:20–12:45 | Demo 1: FSDS idea $\rightarrow$ figure | the six-part prompt |
| 12:45–1:35 | **Lab 1:** your own pipeline | a verified figure |
| 1:35–1:45 | Break | |
| 1:45–2:15 | Demo 2 + **Lab 2a:** a skill | a tested `SKILL.md` |
| 2:15–2:40 | **Lab 2b:** verification drills | audit, referee, deny rule |
| 2:40–3:00 | Discussion + exit ticket | a written position |

# What You Did Before Class

- ✓ Installed Claude Code and signed in
- ✓ Ran the smoke test: `setup_check.txt`
- ✓ Downloaded three FSDS quarters into `fsds/`
- ✓ Watched the Module 1 video
- ✓ Read Brodeur et al. and Kusumegi et al.
- ✓ Wrote one research question

## Check now

`ls fsds/`

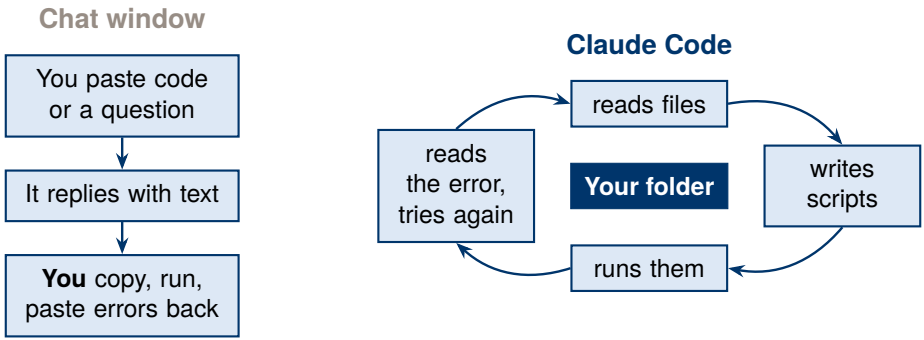
You should see `2016q1`, `2020q1` and `2024q1`, each holding `sub.txt` and `num.txt`.

Missing a quarter? Raise your hand now, not at 12:45.

Block 1 · 12:00–12:20

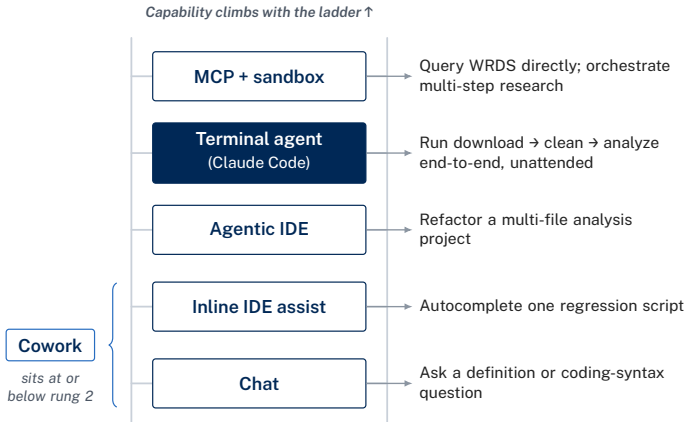
# Framing

# Chat Window vs. Agent




Claude Code uses the Python, R or Stata already on your machine. It has none of its own.

# The Tool Ladder




Claude Code sits on the terminal-agent rung. Pick the rung the task needs.

# Staying in Control: Four Tools

`CLAUDE.md`
project memory, read at the start of every session

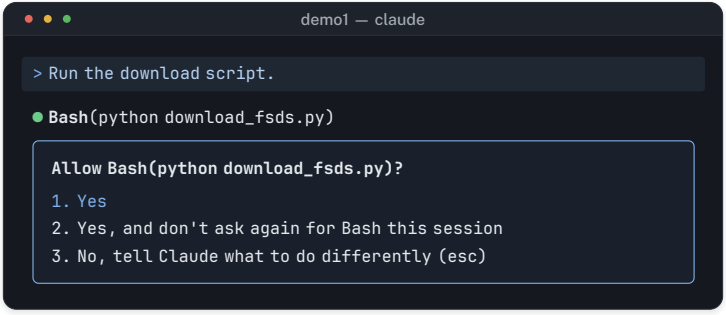
**Permission prompts**
it asks before an edit, a command or a download

**Plan mode**
it writes a plan and changes nothing until you approve

**Esc**
stops it mid-step so you can redirect

Shift+Tab cycles the permission modes, including plan mode. The status bar shows which one you are in.

# What a Permission Prompt Looks Like

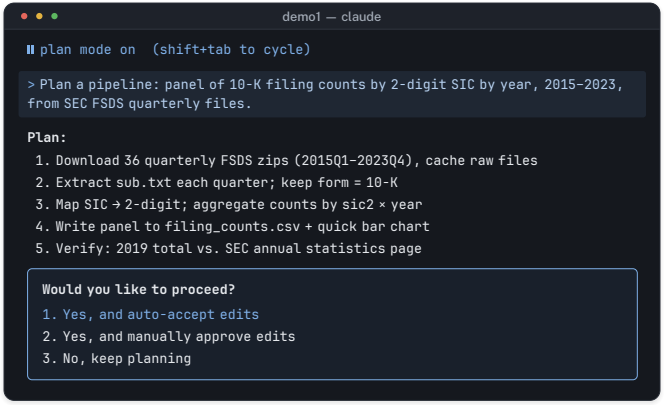



**Manual:** ask every time — safest while trust is new

**Auto:** a classifier reviews each action — pair with git (Module 5)

*Stylized session transcript (not a screenshot).*

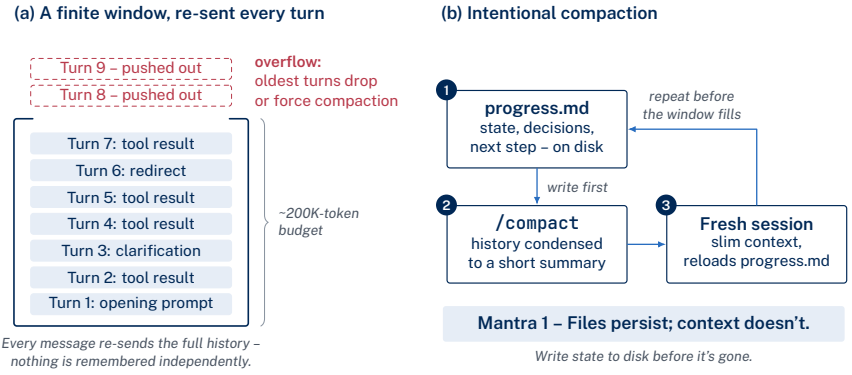
# Plan Mode: Read the Plan First




*Stylized session transcript (not a screenshot).*

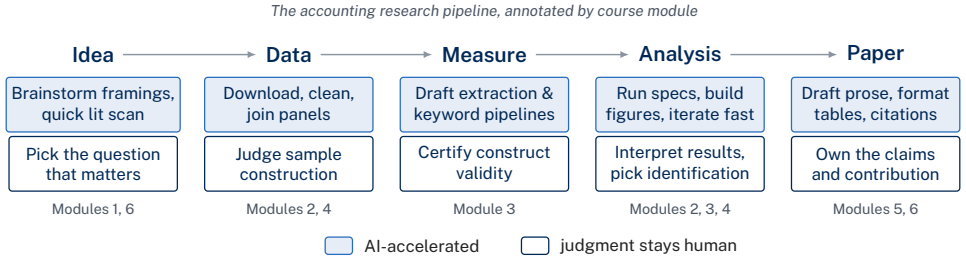
Design before code: review the plan, then execute — or don't.

# Files Persist; Context Doesn't




A new session remembers nothing you did not write to a file. That is why today's labs ask for files, not chat replies.

# Verification Runs Through Everything



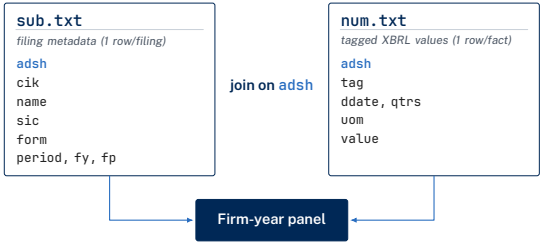

## The standard you already apply to an RA

Check one number against a source Claude did not touch before you trust the rest.

Block 2 · 12:20–12:45

# Live Demo: FSDS Idea to Verified Figure

# SEC FSDS: Two Files, One Key



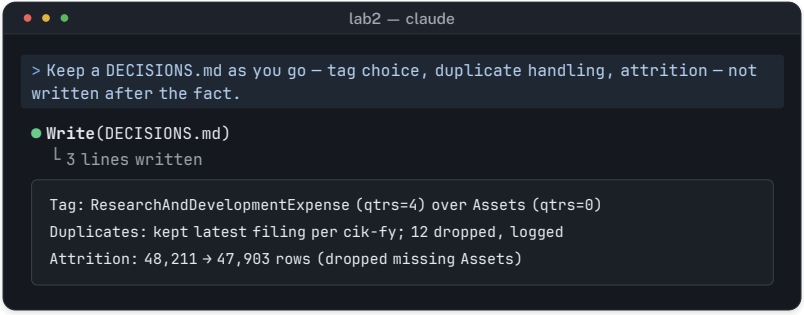

- `sub.txt`: one row per filing
- `num.txt`: one row per tagged value
- Join on `adsh`, and state the row counts before and after
- Free, quarterly, no login

# The Six-Part Opening Prompt

| | |
|---|---|
| **DATA** | which quarters, which two files, which columns |
| **JOIN** | merge on `adsh`; consolidated rows only; counts before and after |
| **MEASURE** | the tags, the `qtrs` value for each, `form` = 10-K |
| **AGGREGATION** | which statistic, grouped by what, over what period |
| **OUTPUT** | figure and table, named in advance |
| **PAPER TRAIL** | named scripts; `DATA_PROFILE.md`, `DECISIONS.md`, `LOG.md` |

Paste all six before any code runs. Your copy is `lab1_opening_prompt.md`.

# The Paper Trail, Written as It Runs

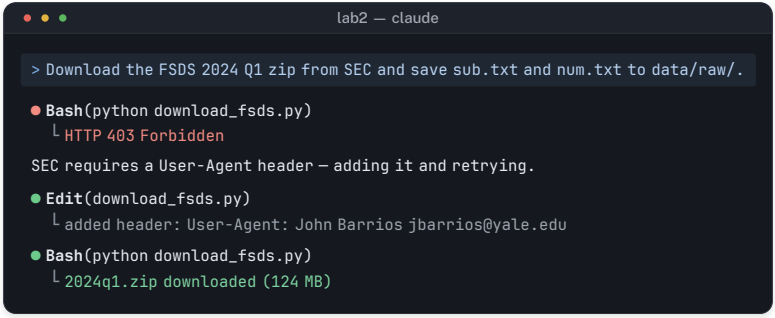



*Stylized session transcript (not a screenshot).*

The paper trail requested up front, written as the pipeline runs — not reconstructed after.

Asked for in the opening prompt, so it is written during the work rather than reconstructed afterwards.

# A Planned Failure: the SEC's 403

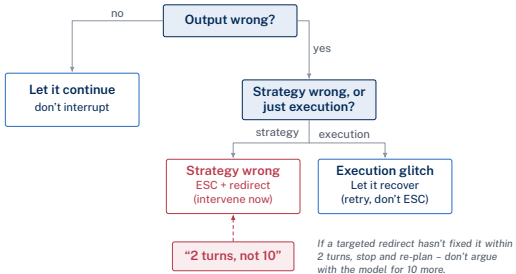



*Stylized session transcript (not a screenshot).*

Autonomous error recovery: read the error, fix the cause, retry — no prompt from you.

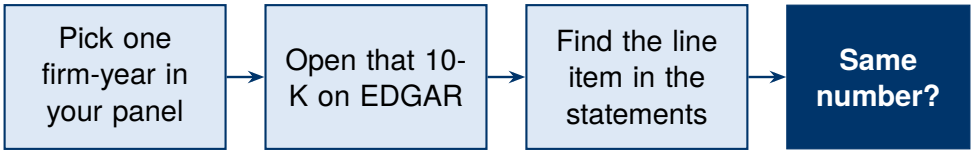
The SEC refuses requests without a User-Agent of the form "Name email". Watch it read the error and fix it.

# When to Step In




- A failed download is execution: let it recover
- A duplicated firm-year is strategy: press Esc and say what to do
- Two redirects without progress: stop and re-plan

# One Number, One 10-K




## Write it down

Firm, fiscal year, your value, the 10-K value, and the page. A figure that looks right is not yet a checked figure.

Block 3 · 12:45–1:35

# Lab 1: Your Own Pipeline

# Lab 1: Start Here

```
cd ~/stanford_lab
claude
```

1. Open `lab1_opening_prompt.md`
2. Fill in your tier
3. Paste the whole prompt
4. Approve the plan, then watch

**Data you already have**

`fsds/2016q1`, `fsds/2020q1`, `fsds/2024q1`

These hold mostly fiscal years 2015, 2019 and 2023. No downloads needed in the room.

50 minutes. Finish with the verify checklist, even if your figure is unfinished.

# Three Tiers: Pick One

**Base**

**Cash holdings**

cash ÷ total assets
median by fiscal year

tag: see lab page

**Standard**

**R&D intensity**

`ResearchAndDevelopmentExpense (qtrs=4)` ÷ `Assets`

median by SIC2 (five largest), by year

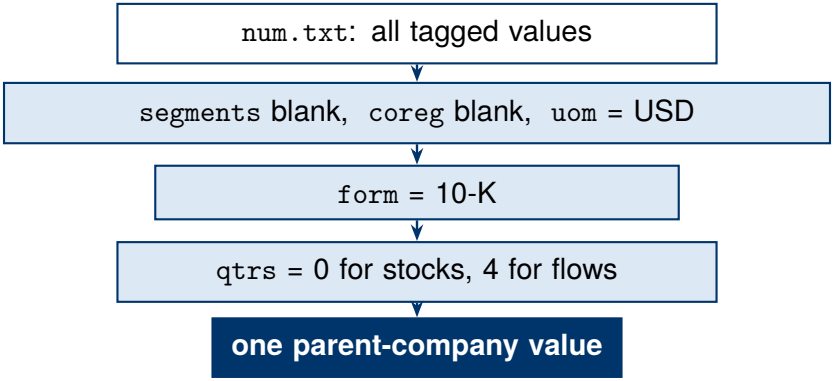
**Stretch**

**Your own question**

same paper trail,
same verify checklist

Choose by how much cleaning judgment you want to practice today, not by ambition.

# The Consolidated-Row Filter




Print an attrition row (n before, n after, reason) at every step.

# Three Traps to Watch For

## Zero vs. missing

A firm with no R&D tag is missing, not zero. Filling it with zero changes the median.

## Duplicate firm-years

A 10-K also reports last year's values. Keep the value for the filing's own period.

## Memory

`num.txt` is large. Read one quarter at a time and keep only the tags you need.

Tag coverage differs by quarter. Measure it in `DATA_PROFILE.md` before you trust a median.

# The Paper-Trail Files

**`DATA_PROFILE.md`**

what one row is; a uniqueness test; tag coverage; rules for duplicates and `10-K/A`

**`DECISIONS.md`**

every choice as it is made: which tag, which filter, why

**`LOG.md`**

a timestamped line for every step, with attrition rows

**before any figure**

Delete the chat. Can someone rebuild your figure from these files and the named scripts?

# Lab 1 Verify Checklist

- ☐ `DATA_PROFILE.md` exists and was written before the first figure
- ☐ Consolidated filter applied; `form` = 10-K; `qtrs` set for each tag
- ☐ Attrition row at every change in sample size
- ☐ Every choice recorded in `DECISIONS.md`
- ☐ Figure and table produced by named scripts that rerun from the README
- ☐ One number checked against the actual 10-K on EDGAR

Full list: `lab1_verify_checklist.md`. Leave an unchecked box unchecked and say why.

# Break

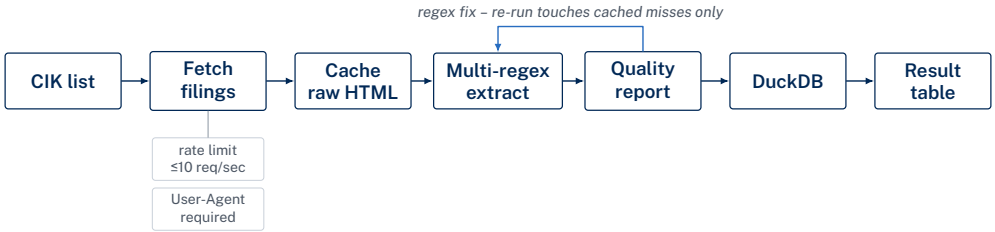
1:35–1:45

Leave Claude running if your pipeline is mid-step.

Block 4 · 1:45–2:15

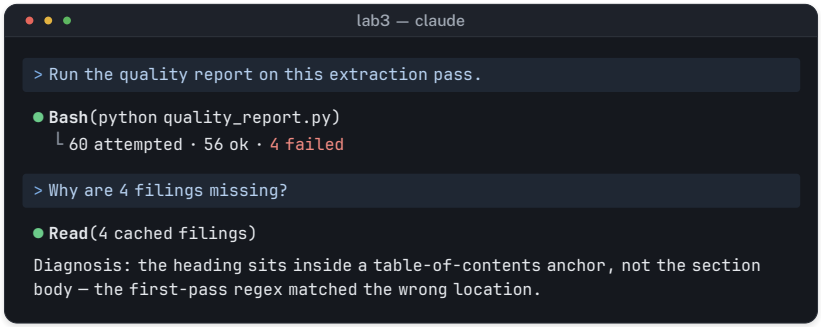
# Demo 2 and Lab 2a: From Pipeline to Skill

# Demo 2: EDGAR Text to Data




- One measure: a keyword count in Item 1A, Risk Factors
- Raw filings cached, so a fixed regex re-runs with no new downloads
- A count is a word frequency, not yet a measure of risk

# The Quality Report

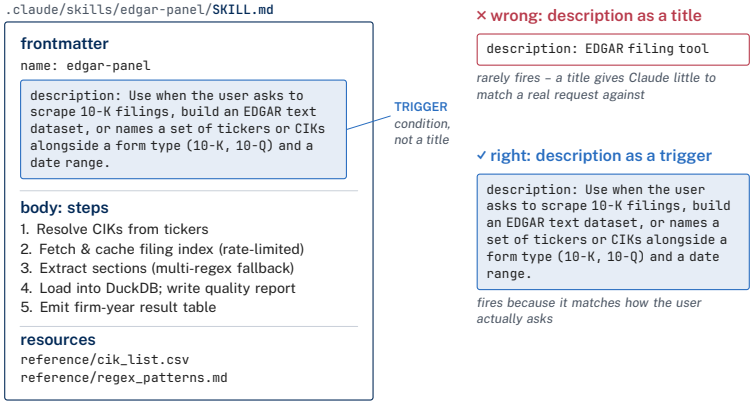



*Stylized session transcript (not a screenshot).*

Illustrative numbers. Read what the quality report flagged before you re-run anything.

An extraction rate needs its denominator, and every failure needs a one-line reason.

# Anatomy of a SKILL.md




Only the description loads every session. The steps load when the skill runs.

# Lab 2a: Turn Lab 1 Into `fsds-panel`

1. Copy `SKILL_skeleton.md` to `.claude/skills/fsds-panel/SKILL.md`
2. Frontmatter on line 1: the file starts with `---`, nothing above it
3. Write the description as a trigger: the phrases a real request uses
4. Steps: your Lab 1 pipeline, in order, with the filter and the checks
5. Test in a fresh session, with a request that does not name the skill

# Title vs. Trigger

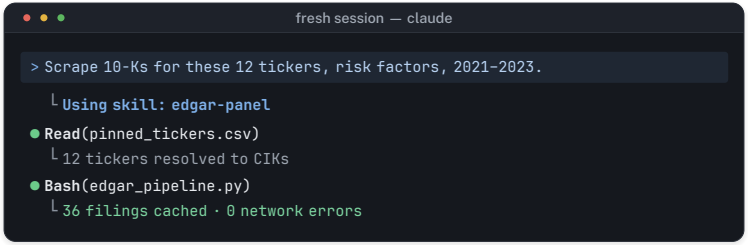
**Rarely fires (a title):**

```
description: FSDS panel tool
```

**Fires (names what you would actually type):**

```
description: Use when the user asks for a firm-year
  panel or a median ratio from SEC Financial Statement
  Data Sets (FSDS), names XBRL tags such as Assets, or
  asks for a measure by fiscal year or SIC code.
```

# Test It in a Fresh Session




*Stylized session transcript (not a screenshot).*

A fresh session, no re-explaining: the description field fires on the request's own words.

Quit, start `claude` again, and ask for a measure in your own words. You should see it load `fsds-panel` by name. If it reads your old scripts instead, rewrite the description.

Block 5 · 2:15–2:40

# Lab 2b: Verification Drills

# Three Drills on Your Own Repo

**1**
**Credential audit**
two checks; both should print nothing

**2**
**Clean-room referee**
a fresh session that has never heard your reasons

**3**
**One deny rule**
written in `settings.json`, then tested

Steps and commands: `lab2_verification_drills.md`. Paste each output into your notes; “it’s clean” is not the deliverable.

# Check 1: Credential Files in History

```
git log --all --full-history --oneline -- '*.env' '*.env.*' '.mcp.json'
```

- Expected output: nothing
- Finds a `.env` file or a variant such as `.env.local`, in any folder, at any point in the history, plus a project-level `.mcp.json`
- Deleting the file now does not remove it from history

# Check 2: Your Actual Key

```
read -s SECRET
git log --all --oneline -S"$SECRET"
git grep -l -F --untracked -e "$SECRET"
```

- Paste the key after `read -s`; it won't echo. Expected output: nothing
- The two searches print commit IDs or file names, never the key
- No API keys in this project? Run Check 1 only

**If either check finds something**

Revoke and rotate the key first. Clean the history second.

# Drill 2: The Clean-Room Referee

```
mkdir -p ~/referee_room/paper ~/referee_room/.claude
cp ~/stanford_lab/summary.md ~/stanford_lab/*.png ~/referee_room/paper/
echo '{"permissions":{"deny":["Edit(./paper/**)"]}}' \
  > ~/referee_room/.claude/settings.json
cd ~/referee_room && claude
```

- Empty folder: no `CLAUDE.md`, no `DECISIONS.md`, no chat history
- The deny rule, not the prompt, keeps it from editing the text
- Ask for the three most serious objections, written to `review/`

# Why a Separate Room

**Same session**

has heard every reason you gave for each filter

tends to accept your design

**Clean room**

sees only the text a reader would see

has to rebuild the sample from it

## A referee that agrees with you

If the report is mostly praise, or backs down when you object, ask again for the three most serious problems. Answer it in the report file, not in the chat.

# Drill 3: One Deny Rule, Tested

In `.claude/settings.json`:

```
{
  "permissions": {
    "deny": [
      "Edit(./fsds/**)"
    ]
  }
}
```

Raw data stays read-only.

**Test it**

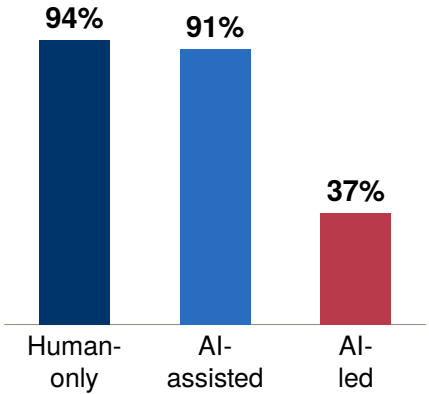
1. Start a new session in the project
2. Run `/permissions` and find the rule
3. Ask Claude to edit a file in `fsds/`
4. It should refuse. Paste what it said

Deny rules hold in every mode. They cover Claude's file tools, not every shell command, so they are not a sandbox.

Block 6 · 2:40–3:00

# Discussion and Exit Ticket

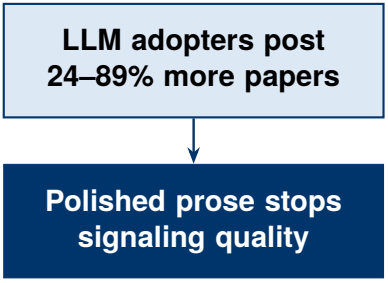
# Brodeur et al. (2026), *PNAS*




share of teams that reproduced the paper

- 288 researchers in 103 teams reproducing published papers
- AI-assisted teams matched human-only teams on reproduction
- They caught fewer of the major coding errors
- AI-led teams reproduced 37%

# Kusumegi et al. (2025), *Science*




- Preprints before and after authors start using LLMs
- The range is across the preprint servers studied
- Among adopters, more complex writing no longer goes with a better paper

If writing gets cheap and stops signaling quality, readers and referees have to look somewhere else.

# Three Questions

1. **Knowledge vs. learning.** Which step of today's pipeline would you not let an agent do for you in your first paper, because doing it is how you learn the data?
2. **Who did the work.** Brodeur et al.'s AI-assisted teams reproduced as often but missed more major errors. Which item on your verify checklist would have caught one?
3. **Returns to research.** If anyone can build today's figure in 50 minutes and prose no longer signals quality, what stays scarce in your work?

# Exit Ticket (`exit_ticket.md`)

| | | | |
|---|---|---|---|
| **One thing that broke** and what fixed it | **One thing you will verify next time** | **The research task you will delegate** | **The one you won't** |

Write it before you leave. Your answers to the last two are your position on what this changes for your research.

# Where to Go Next

## The full course

Six modules: EDGAR text, WRDS at scale, writing and verification, and a seminar on the research market.

`johnmbarrios.com/claude-code-accounting`

## WRDS MCP Setup

Connect Claude Code to your own WRDS account, with Duo, and keep the password out of your repo.

Setup → WRDS MCP Setup

## Skills catalog

Skills built for accounting research. Read each one before you install it.

Skills tab on the course site

Access code and links: the *After class* page of this session's site.